

Skill – 16

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano background_jobs.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc background_jobs.c -o background_jobs
himasree@LAPTOP-41COT5AA:~/OSLab$ ./background_jobs
=====
BACKGROUND JOB MANAGER
=====

-----
1. Start Background Job
2. Show Job Table
3. Check Job Status
4. Exit
-----
Enter choice: 1
Enter execution time in seconds: 10
[1] 629 Running sleep background task
Parent continues immediately.

-----
1. Start Background Job
2. Show Job Table
3. Check Job Status
4. Exit
-----
Enter choice:
Child process 629 started.
Running background task for 10 seconds...
Background task 629 finished.
|
```

```
=====
JOB TABLE
=====
No active background jobs.

-----
1. Start Background Job
2. Show Job Table
3. Check Job Status
4. Exit
-----
Enter choice: 1
Enter execution time in seconds: 2
[2] 630 Running sleep background task
Parent continues immediately.

-----

Child process 630 started.
1. Start Background Job
2. Show Job Table
Running background task for 2 seconds...
3. Check Job Status
4. Exit
-----
Enter choice: Background task 630 finished.
|
```

```

himasree@LAPTOP-41COT5AA:~/OSLab$ cat background_jobs.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef enum
{
    RUNNING,
    COMPLETED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    JobState state;
    char command[100];
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

```

```

/* =====
ADD JOB TO JOB TABLE
===== */

void add_job(pid_t pid, pid_t pgid, const char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    job_table[job_count].job_id = next_job_id++;
    job_table[job_count].pid = pid;
    job_table[job_count].pgid = pgid;
    job_table[job_count].state = RUNNING;

    strncpy(job_table[job_count].command,
            command,
            sizeof(job_table[job_count].command) - 1);

    job_table[job_count].command[
        sizeof(job_table[job_count].command) - 1
    ] = '\0';

    printf("[%d] %d Running %s\n",
        job_table[job_count].job_id,
        pid,
        job_table[job_count].command);
}

```

```

/* =====
REMOVE COMPLETED JOB
===== */

void remove_job(int index)
{
    for (int i = index; i < job_count - 1; i++)
    {
        job_table[i] = job_table[i + 1];
    }

    job_count--;
}

/* =====
CHECK BACKGROUND JOBS
===== */

void check_jobs()
{
    int status;

    for (int i = 0; i < job_count; i++)
    {
        pid_t result = waitpid(
            job_table[i].pid,
            &status,
            WNOHANG
        );
    }
}

```

```

/* =====
DISPLAY JOB TABLE
===== */

void show_jobs()
{
    check_jobs();

    printf("\n=====\\n");
    printf("          JOB TABLE\\n");
    printf("=====\\n");

    if (job_count == 0)
    {
        printf("No active background jobs.\\n");
        return;
    }

    printf("%-8s %-10s %-10s %-12s %s\\n",
           "Job ID",
           "PID",
           "PGID",
           "State",
           "Command");

    printf("-----\\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("%-8d %-10d %-10d %-12s %s\\n",

```

```

/* =====
START BACKGROUND JOB
===== */

void start_background_job(int seconds)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return;
    }

    /*
    CHILD PROCESS
    */

    if (pid == 0)
    {
        /*
        Create a new process group.
        The child becomes the group leader.
        */

        setpgid(0, 0);

        printf("\\nChild process %d started.\\n", getpid());

```

```

        for (int i = 0; i < job_count; i++)

```

```

int seconds;

printf("=====\n");
printf("      BACKGROUND JOB MANAGER\n");
printf("=====\n");

while (1)
{
    /*
     * Check whether any background jobs
     * have completed.
     */

    check_jobs();

    printf("\n-----\n");
    printf("1. Start Background Job\n");
    printf("2. Show Job Table\n");
    printf("3. Check Job Status\n");
    printf("4. Exit\n");
    printf("-----\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:

            printf("Enter execution time in seconds: ");

```

```

            check_jobs();

            printf("\nJob status updated.\n");

            break;

        case 4:

            /*
             * Check once before exiting.
             */

            check_jobs();

            printf("\nExiting job manager.\n");

            return 0;

        default:

            printf("Invalid choice.\n");

    }

    return 0;
}

```

hima5ree@LAPTOP-#1C0T54A:~/OSLab\$ |